



FASHION STORE

MICROSERVICE ARCHITECTURE

Project Plan & Technical Blueprint

Java 21 • Spring Boot • PostgreSQL • Docker
JWT / OAuth2 • Spring Cloud Gateway • Resilience4j • MapStruct
Flyway • OpenAPI • RestClient • Common Library

Phiên bản 1.0 | Tháng 6, 2026

Mục Lục

1. Tổng quan dự án	3
2. Kiến trúc tổng thể	4
3. Cấu trúc Maven Multi-module	6
4. Chi tiết từng Microservice	7
5. Common Library	14
6. Database Schema	15
7. Security & Authentication	18
8. Design Patterns áp dụng	20
9. Docker & Docker Compose	22
10. API Gateway Routing	24
11. Resilience & Error Handling	25
12. Lộ trình phát triển (Roadmap)	27
13. Cấu trúc thư mục chi tiết	29
14. Checklist hoàn thành	31

1. Tổng Quan Dự Án

1.1 Mô tả dự án

Fashion Store là hệ thống bán quần áo trực tuyến được xây dựng theo kiến trúc Microservice. Hệ thống cho phép người dùng duyệt sản phẩm, quản lý giỏ hàng, đặt hàng và thanh toán. Mỗi chức năng được tách thành một service độc lập với database riêng.

1.2 Stack công nghệ

Nhóm	Công nghệ	Mục đích
Backend Runtime	Java 21 + Spring Boot 3.x	Core framework, Virtual Threads
Build Tool	Maven Multi-module	Quản lý dependency tập trung
API Gateway	Spring Cloud Gateway	Single entry point, routing
Security	Spring Security + OAuth2 RS	Xác thực & phân quyền
Authentication	Identity Service + Spring Auth Server	Cấp JWT, refresh token
Token Format	JWT RSA (RS256)	Private key ký, Public key xác minh
Database	PostgreSQL (per service)	RDBMS, mỗi service 1 DB
ORM	Spring Data JPA + Hibernate	Entity mapping, query
Migration	Flyway	Quản lý schema version
Service Call	RestClient (Spring 6)	Synchronous HTTP gọi service
Resilience	Resilience4j	Retry, CircuitBreaker, Fallback
Mapping	MapStruct	Entity ↔ DTO mapping
Validation	Jakarta Validation	Input validation
Logging	SLF4J + Logback	Structured logging
API Docs	OpenAPI 3 / Swagger UI	Auto-generate API docs
Container	Docker + Docker Compose	Containerization, orchestration
Common Lib	common-library module	Shared response, exception, utils

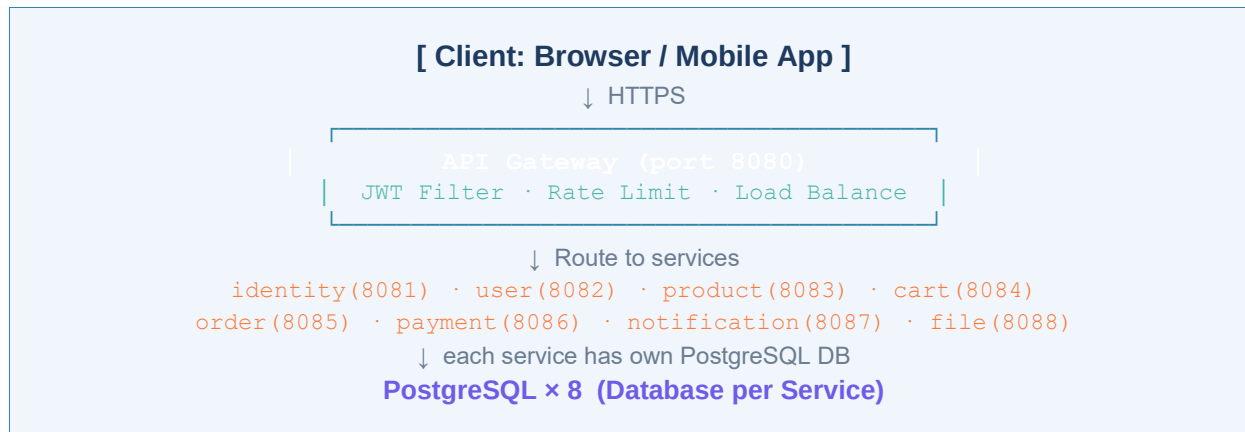
1.3 Danh sách Microservices

Service	Port	Database	Chức năng chính
api-gateway	8080	N/A	Routing, auth filter, rate limiting
identity-service	8081	identity_db	Đăng ký, đăng nhập, cấp JWT, quản lý token
user-service	8082	user_db	Profile người dùng, địa chỉ, thông tin cá nhân
product-service	8083	product_db	Sản phẩm, danh mục, kích cỡ, màu sắc, tồn kho
cart-service	8084	cart_db	Giỏ hàng, thêm/xóa/cập nhật sản phẩm
order-service	8085	order_db	Đặt hàng, lịch sử đơn, trạng thái đơn hàng
payment-service	8086	payment_db	Xử lý thanh toán, lịch sử giao dịch

Service	Port	Database	Chức năng chính
notification-service	8087	notification_db	Email, SMS, push notification
file-service	8088	file_db	Upload/download ảnh sản phẩm

2. Kiến Trúc Tổng Thể

2.1 Architecture Overview



2.2 Request Flow

Bước	Component	Hành động	Ghi chú
1	Client	Gửi request với Bearer JWT	Token trong Authorization header
2	API Gateway	Kiểm tra JWT signature bằng Public Key	RSA RS256, không cần gọi identity-service
3	API Gateway	Extract claims, forward X-User-Id header	userId, roles truyền xuống service
4	Target Service	Nhận request, xác thực từ header	Spring OAuth2 Resource Server
5	Target Service	Xử lý business logic	Gọi DB, gọi service khác nếu cần
6	Service → Service	RestClient + JWT forward	Resilience4j bảo vệ circuit
7	Response	Trả về ApiResponse chuẩn	Từ common-library

2.3 Token Flow (JWT RSA)

Step	Actor	Action
1. Login	Client → identity-service	POST /auth/login với credentials
2. Issue Token	identity-service	Ký JWT bằng RSA Private Key, trả access + refresh token
3. Store Token	Client	Lưu access token (memory), refresh token (httpOnly cookie)
4. API Call	Client → API Gateway	Gửi request với Bearer access token
5. Verify	API Gateway	Verify chữ ký bằng RSA Public Key từ JWKS endpoint
6. Route	API Gateway → Service	Forward request với claims trong header
7. Refresh	Client → identity-service	Khi access token hết hạn, dùng refresh token để lấy token mới

3. Cấu Trúc Maven Multi-module

3.1 Module hierarchy

fashion-store/	← Parent POM (root)
├─ common-library/	← Shared code (response, exception, utils)
├─ api-gateway/	← Spring Cloud Gateway
├─ identity-service/	← Auth, JWT issuance
├─ user-service/	← User profile management
├─ product-service/	← Product catalog, inventory
├─ cart-service/	← Shopping cart
├─ order-service/	← Order management
├─ payment-service/	← Payment processing
├─ notification-service/	← Email/SMS/Push
├─ file-service/	← File upload/download
└─ docker-compose.yml	← All services + databases

3.2 Parent POM dependencies chính

Dependency / Plugin	Version	Scope / Note
spring-boot-starter-parent	3.3.x	BOM parent
spring-cloud-dependencies	2023.0.x	Gateway, LoadBalancer
mapstruct	1.5.5.Final	Compile-time mapper
lombok	1.18.x	Compile-time code gen
resilience4j-spring-boot3	2.x	Circuit Breaker, Retry
flyway-core	10.x	DB migration
openapi-webmvc-ui	2.x	Swagger UI auto-gen
jackson-databind	BOM	JSON serialization
maven-compiler-plugin	3.12.x	annotationProcessorPaths: mapstruct + lombok

4. Chi Tiết Từng Microservice

4.1 identity-service (port 8081)

Service trung tâm xác thực, chịu trách nhiệm đăng ký, đăng nhập và cấp JWT token.

Package	Class/File	Vai trò
controller	AuthController	POST /auth/register, /auth/login, /auth/refresh, /auth/logout
controller	IntrospectController	POST /auth/introspect (kiểm tra token validity)
service	AuthService	Business logic: validate, hash password, tạo token
service	TokenService	Tạo/verify JWT bằng RSA key, quản lý refresh token
repository	UserCredentialRepository	JPA repo cho UserCredential entity
repository	RefreshTokenRepository	Lưu refresh token với expiry
entity	UserCredential	id, username, password(hashed), roles, status
entity	RefreshToken	token, userId, expiredAt, revoked
security	SecurityConfig	Permit /auth/**, cấu hình JWT decoder
config	RsaKeyConfig	Load RSA private/public key từ .pem file
migration	V1__init_auth.sql	Flyway: tạo bảng user_credentials, refresh_tokens

4.2 user-service (port 8082)

Quản lý thông tin cá nhân, địa chỉ giao hàng của người dùng.

Package	Class/File	Vai trò
controller	UserController	GET /users/me, PUT /users/me, GET /users/{id} (admin)
controller	AddressController	CRUD địa chỉ: POST/GET/PUT/DELETE /users/me/addresses
service	UserService	Logic cập nhật profile, validate uniqueness
service	AddressService	Quản lý danh sách địa chỉ, chọn địa chỉ mặc định
repository	UserProfileRepository	JPA repo cho UserProfile
repository	AddressRepository	JPA repo cho Address
entity	UserProfile	id, userId(from JWT), fullName, phone, avatar, createdAt
entity	Address	id, userId, street, ward, district, province, isDefault
mapper	UserMapper	MapStruct: UserProfile ↔ UserProfileDto
client	IdentityServiceClient	RestClient gọi identity-service (nếu cần verify)
migration	V1__init_user.sql	Flyway: tạo bảng user_profiles, addresses

4.3 product-service (port 8083)

Quản lý catalog sản phẩm, danh mục, biến thể (size/màu), tồn kho.

Package	Class/File	Vai trò
controller	ProductController	CRUD sản phẩm, tìm kiếm, lọc theo category/size/color
controller	CategoryController	CRUD danh mục sản phẩm (tree structure)
controller	InventoryController	Kiểm tra & cập nhật tồn kho theo variant
service	ProductService	Logic tạo/cập nhật sản phẩm, quản lý variants
service	CategoryService	Xây dựng category tree, breadcrumb
service	InventoryService	Check stock, reserve stock khi tạo order
entity	Product	id, name, desc, basePrice, categoryId, status, images
entity	ProductVariant	id, productId, size, color, sku, price, stockQty
entity	Category	id, name, parentId, slug, imageUrl
entity	ProductImage	id, productId, imageUrl, isPrimary, sortOrder
mapper	ProductMapper	MapStruct: Product ↔ ProductDto, Variant ↔ VariantDto
migration	V1__init_product.sql	Tạo bảng products, product_variants, categories, product_images

4.4 cart-service (port 8084)

Quản lý giỏ hàng per user, tự động xóa item hết hàng.

Package	Class/File	Vai trò
controller	CartController	GET /cart, POST /cart/items, PUT /cart/items/{id}, DELETE /cart/items/{id}
service	CartService	Add/update/remove items, tính total, validate stock
service	CartValidationService	Gọi product-service kiểm tra stock và giá hiện tại
entity	Cart	id, userId, createdAt, updatedAt
entity	CartItem	id, cartId, productId, variantId, quantity, priceSnapshot
client	ProductServiceClient	RestClient gọi product-service kiểm tra variant, giá, tồn kho
mapper	CartMapper	Cart/CartItem ↔ CartDto/CartItemDto
migration	V1__init_cart.sql	Tạo bảng carts, cart_items

4.5 order-service (port 8085)

Xử lý đặt hàng, quản lý trạng thái đơn hàng theo state machine.

Package	Class/File	Vai trò
controller	OrderController	POST /orders (checkout), GET /orders, GET /orders/{id}, PUT /orders/{id}/cancel
service	OrderService	Tạo order từ cart, validate, tính tổng tiền

Package	Class/File	Vai trò
service	OrderStatusService	State machine: PENDING→CONFIRMED→SHIPPING→DELIVERED/CANCELLED
client	CartServiceClient	Lấy cart items khi checkout
client	ProductServiceClient	Reserve/release stock
client	PaymentServiceClient	Khởi tạo payment session
client	UserServiceClient	Lấy địa chỉ giao hàng
entity	Order	id, userId, status, totalAmount, shippingAddressSnapshot, createdAt
entity	OrderItem	id, orderId, productId, variantId, qty, unitPrice, productSnapshot
mapper	OrderMapper	Order/OrderItem ↔ OrderDto/OrderItemDto
migration	V1__init_order.sql	Tạo bảng orders, order_items

4.6 payment-service (port 8086)

Xử lý thanh toán, tích hợp payment gateway, quản lý giao dịch.

Package	Class/File	Vai trò
controller	PaymentController	POST /payments/initiate, GET /payments/{orderId}, POST /payments/webhook
service	PaymentService	Khởi tạo payment, xử lý kết quả, rollback khi thất bại
service	WebhookService	Nhận callback từ payment gateway (VNPay/Momo/Stripe)
client	OrderServiceClient	Cập nhật trạng thái order sau thanh toán
entity	Payment	id, orderId, userId, amount, method, status, transactionId
entity	PaymentLog	id, paymentId, event, payload, createdAt
mapper	PaymentMapper	Payment ↔ PaymentDto
migration	V1__init_payment.sql	Tạo bảng payments, payment_logs

4.7 notification-service (port 8087)

Gửi thông báo qua email/SMS. Trong giai đoạn đầu nhận request đồng bộ, sau nâng cấp sang Kafka.

4.8 file-service (port 8088)

Upload ảnh sản phẩm, avatar người dùng. Lưu file vào local storage hoặc S3, trả về URL public.

5. Common Library

5.1 Cấu trúc common-library module

common-library là module dùng chung không có main class, chỉ chứa code được import bởi các service khác.

Package	Class	Nội dung
response	ApiResponse<T>	Generic wrapper: code, message, data, timestamp
response	PageResponse<T>	Pagination wrapper: content, page, size, totalElements, totalPages
exception	AppException	RuntimeException với ErrorCode enum
exception	ErrorCode	Enum: USER_NOT_FOUND, PRODUCT_NOT_FOUND, INVALID_TOKEN, UNAUTHORIZED, ...
exception	GlobalExceptionHandler	@RestControllerAdvice xử lý exception toàn cục
exception	ServiceUnavailableException	Khi service downstream unavailable
dto	UserContextDto	userId, username, roles - extract từ JWT/header
utils	SecurityUtils	Lấy current user từ SecurityContext hoặc header
utils	DateTimeUtils	Xử lý LocalDateTime, format, parse
utils	StringUtils	Slug generation, text normalize
config	JacksonConfig	Global Jackson config: LocalDate serializer, null exclude
annotation	@CurrentUser	Custom annotation để inject UserContextDto vào controller
constant	AppConstants	PAGE_SIZE, MAX_FILE_SIZE, TOKEN_PREFIX, ...

5.2 ApiResponse template

```

@Builder @Getter
public class ApiResponse<T> {
    private int code;                // 2000 = success, 4xxx = client error, 5xxx =
server error
    private String message;
    private T data;
    private LocalDateTime timestamp;

    public static <T> ApiResponse<T> success(T data) {
        return ApiResponse.<T>builder()
            .code(2000).message("Success").data(data)
            .timestamp(LocalDateTime.now()).build();
    }
    public static ApiResponse<Void> error(ErrorCode errorCode) { ... }
}

```

6. Database Schema

6.1 identity_db

Bảng	Cột chính	Index / Constraint
user_credentials	id UUID PK, username VARCHAR(50) UNIQUE, password_hash VARCHAR(255), roles VARCHAR(255), status, created_at	UNIQUE(username), INDEX(status)
refresh_tokens	id UUID PK, token VARCHAR(512) UNIQUE, user_id UUID FK, expired_at TIMESTAMP, revoked BOOLEAN	INDEX(user_id), INDEX(token), INDEX(expired_at)

6.2 user_db

Bảng	Cột chính	Index / Constraint
user_profiles	id UUID PK, user_id UUID UNIQUE, full_name, phone VARCHAR(20), avatar_url, date_of_birth, gender, created_at, updated_at	UNIQUE(user_id), INDEX(phone)
addresses	id UUID PK, user_id UUID FK, full_name, phone, street, ward, district, province, is_default BOOLEAN, created_at	INDEX(user_id), PARTIAL INDEX ON is_default

6.3 product_db

Bảng	Cột chính	Index / Constraint
categories	id UUID PK, name VARCHAR(100), slug VARCHAR(100) UNIQUE, parent_id UUID NULLABLE FK(self), image_url, sort_order	UNIQUE(slug), INDEX(parent_id)
products	id UUID PK, name VARCHAR(255), description TEXT, base_price NUMERIC(12,2), category_id UUID FK, status ENUM, created_at, updated_at	INDEX(category_id), INDEX(status), FULLTEXT(name)
product_variants	id UUID PK, product_id UUID FK, size VARCHAR(10), color VARCHAR(50), sku VARCHAR(100) UNIQUE, price NUMERIC(12,2), stock_quantity INT	UNIQUE(sku), INDEX(product_id), INDEX(size,color)
product_images	id UUID PK, product_id UUID FK, image_url TEXT, is_primary BOOLEAN, sort_order INT	INDEX(product_id)

6.4 cart_db

Bảng	Cột chính	Index / Constraint
carts	id UUID PK, user_id UUID UNIQUE, created_at, updated_at	UNIQUE(user_id)
cart_items	id UUID PK, cart_id UUID FK, product_id UUID, variant_id UUID, quantity INT, price_snapshot NUMERIC(12,2)	INDEX(cart_id), UNIQUE(cart_id, variant_id)

6.5 order_db

Bảng	Cột chính	Index / Constraint
orders	id UUID PK, user_id UUID, status ENUM(PENDING/CONFIRMED/SHIPPING/DELIVERED/CANCELLED), total_amount NUMERIC(12,2), shipping_address JSONB, note TEXT, created_at, updated_at	INDEX(user_id), INDEX(status), INDEX(created_at DESC)
order_items	id UUID PK, order_id UUID FK, product_id UUID, variant_id UUID, quantity INT, unit_price NUMERIC(12,2), product_snapshot JSONB	INDEX(order_id)

6.6 payment_db

Bảng	Cột chính	Index / Constraint
payments	id UUID PK, order_id UUID UNIQUE, user_id UUID, amount NUMERIC(12,2), method ENUM, status ENUM, transaction_id VARCHAR(255), gateway_response JSONB, created_at	UNIQUE(order_id), INDEX(user_id), INDEX(transaction_id)
payment_logs	id UUID PK, payment_id UUID FK, event VARCHAR(100), payload JSONB, created_at	INDEX(payment_id), INDEX(created_at)

7. Security & Authentication

7.1 JWT RSA Key Setup

Mỗi môi trường (dev/staging/prod) có cặp RSA key riêng. Private key chỉ có identity-service. Public key được expose qua JWKS endpoint để Gateway và các service verify.

File	Vị trí	Mục đích
private.pem	identity-service/src/main/resources/certs/	Ký JWT (chỉ identity-service dùng)
public.pem	identity-service/src/main/resources/certs/	Verify JWT (expose qua JWKS)
application.yml	Mỗi service	spring.security.oauth2.resourceserver.jwt.jwk-set-uri: http://identity-service:8081/.well-known/jwks.json

7.2 SecurityConfig per service

Service	Public endpoints (permit all)	Protected endpoints
identity-service	/auth/register, /auth/login, /auth/refresh, /.well-known/jwks.json	Mọi endpoint khác cần JWT
user-service	Không có	Tất cả cần JWT
product-service	GET /products/**, GET /categories/**	POST/PUT/DELETE cần ADMIN role
cart-service	Không có	Tất cả cần JWT (USER role)
order-service	Không có	Tất cả cần JWT
payment-service	/payments/webhook (HMAC verify riêng)	Tất cả cần JWT
file-service	GET /files/** (public images)	POST /files/ cần JWT
api-gateway	/auth/register, /auth/login	Tất cả route khác filter JWT

7.3 Authentication Context Pattern

Mỗi service đọc thông tin user hiện tại từ SecurityContext hoặc header do Gateway forward.

```
// Trong common-library
public class SecurityUtils {
    public static String getCurrentUserId() {
        return SecurityContextHolder.getContext()
            .getAuthentication().getName(); // = subject claim
    }
    public static boolean hasRole(String role) {
        return SecurityContextHolder.getContext()
            .getAuthentication().getAuthorities()
            .stream().anyMatch(a -> a.getAuthority().equals("ROLE_" + role));
    }
}

// Trong Controller
@GetMapping("/me")
public ApiResponse<UserProfileDto> getMyProfile() {
```

```
String userId = SecurityUtils.getCurrentUserId();  
return ApiResponse.success(userService.getByUserId(userId));  
}
```

8. Design Patterns Áp Dụng

8.1 Patterns và implementation

Pattern	Áp dụng tại	Cách implement	Mục đích
API Gateway	api-gateway	Spring Cloud Gateway với RouteLocator, JwtAuthFilter	Single entry point
Database per Service	Mọi service	Mỗi service có application.yml riêng, datasource riêng, Flyway migration riêng	Loose coupling, independent scaling
Shared Kernel	common-library	Maven module được import bởi tất cả service	DRY, consistent response format
DTO Pattern	Mọi service	Request/Response DTO tách biệt với Entity, chỉ expose DTO qua controller	Encapsulation, API stability
Mapper Pattern	Mọi service	MapStruct interface với @Mapper annotation, compile-time generation	Type-safe, boilerplate-free mapping
Repository Pattern	Mọi service	JpaRepository extend với custom query methods	Abstraction over data access
Service Client	cart, order, payment	RestClient bean với baseUrl, timeout, headers preset	Reusable HTTP client per service
Circuit Breaker	cart, order, payment	Resilience4j @CircuitBreaker với fallback method	Fail fast, cascade prevention
Retry Pattern	cart, order	Resilience4j @Retry với exponential backoff	Handle transient failures
Fallback Pattern	cart-service	Trả CartDto với cảnh báo khi product-service down	Graceful degradation
Auth Context	Mọi service	SecurityUtils.getCurrentUserId() từ JWT principal	Stateless user identification
Builder Pattern	Mọi service	Lombok @Builder trên Entity và DTO	Immutable, readable object creation
Saga (tương lai)	order+payment	Choreography qua Kafka events	Distributed transaction
Outbox (tương lai)	order-service	Bảng outbox_events, polling publisher	Reliable event publishing
Event-driven (tương lai)	order, payment, notification	Kafka topic: order.created, payment.completed	Async communication

8.2 RestClient + Resilience4j pattern

```

@Service
public class ProductServiceClient {
    private final RestClient restClient;

    public ProductServiceClient(RestClient.Builder builder,
                               @Value("${service.product.url}") String baseUrl) {
        this.restClient = builder.baseUrl(baseUrl).build();
    }

    @CircuitBreaker(name = "productService", fallbackMethod = "getVariantFallback")
    @Retry(name = "productService")

```

```
public ProductVariantDto getVariant(UUID variantId, String token) {
    return restClient.get()
        .uri("/api/v1/products/variants/{id}", variantId)
        .header("Authorization", "Bearer " + token)
        .retrieve()
        .body(ProductVariantDto.class);
}

private ProductVariantDto getVariantFallback(UUID id, String token, Exception ex) {
    return ProductVariantDto.unavailable(id); // Fallback response
}
}
```


9. Docker & Docker Compose

9.1 Dockerfile template cho mỗi service

```
# Stage 1: Build
FROM maven:3.9-eclipse-temurin-21-alpine AS build
WORKDIR /app
COPY pom.xml .
COPY common-library/ common-library/
COPY identity-service/ identity-service/
RUN mvn -pl common-library,identity-service -am package -DskipTests

# Stage 2: Run
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/identity-service/target/*.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "-Dspring.profiles.active=docker", "app.jar"]
```

9.2 Docker Compose services

Service	Image	Port	Depends On
postgres-identity	postgres:16-alpine	5432	-
postgres-user	postgres:16-alpine	5433	-
postgres-product	postgres:16-alpine	5434	-
postgres-cart	postgres:16-alpine	5435	-
postgres-order	postgres:16-alpine	5436	-
postgres-payment	postgres:16-alpine	5437	-
postgres-notification	postgres:16-alpine	5438	-
postgres-file	postgres:16-alpine	5439	-
identity-service	fashion/identity:latest	8081	postgres-identity
user-service	fashion/user:latest	8082	postgres-user, identity-service
product-service	fashion/product:latest	8083	postgres-product
cart-service	fashion/cart:latest	8084	postgres-cart, product-service
order-service	fashion/order:latest	8085	postgres-order, cart, product, payment
payment-service	fashion/payment:latest	8086	postgres-payment, order-service
notification-service	fashion/notification:latest	8087	postgres-notification
file-service	fashion/file:latest	8088	postgres-file
api-gateway	fashion/gateway:latest	8080	Tất cả service trên

9.3 Health check & Networks

Tất cả service join network "fashion-network" (bridge). Mỗi service dùng health check:

- healthcheck: test: ["CMD", "wget", "-q", "--spider", "http://localhost:{PORT}/actuator/health"]

- interval: 30s, timeout: 10s, retries: 3, start_period: 60s
- Các service downstream chỉ start sau khi upstream healthy

10. API Gateway Routing

10.1 Route configuration

Route ID	Path Pattern	Forward to	Filter
identity-route	/api/v1/auth/**	identity-service:8081	Không filter JWT (public)
user-route	/api/v1/users/**	user-service:8082	JwtAuthFilter + forward userId
product-route	/api/v1/products/**	product-service:8083	JwtAuthFilter (optional cho GET)
category-route	/api/v1/categories/**	product-service:8083	JwtAuthFilter (optional cho GET)
cart-route	/api/v1/cart/**	cart-service:8084	JwtAuthFilter (required)
order-route	/api/v1/orders/**	order-service:8085	JwtAuthFilter (required)
payment-route	/api/v1/payments/**	payment-service:8086	JwtAuthFilter (required)
file-route	/api/v1/files/**	file-service:8088	JwtAuthFilter (required cho upload)
swagger-route	/v3/api-docs/**	Các service	Aggregate Swagger docs

10.2 Custom Gateway Filters

Filter	Order	Mô tả
JwtAuthenticationFilter	1	Verify JWT signature bằng public key. Reject với 401 nếu invalid/expired
UserContextPropagationFilter	2	Extract userId, roles từ JWT claims, thêm vào X-User-Id, X-User-Roles header
RequestLoggingFilter	3	Log request: method, path, userId, duration
RateLimitingFilter	4	Redis-based rate limiting per IP hoặc per userId
CorrelationIdFilter	5	Thêm X-Correlation-Id header cho distributed tracing

11. Resilience & Error Handling

11.1 Resilience4j configuration

Config	Giá trị	Service áp dụng	Giải thích
CircuitBreaker - failureRateThreshold	50%	cart, order, payment	Mở CB khi 50% request fail
CircuitBreaker - slowCallDurationThreshold	2000ms	cart, order	Request > 2s coi là slow call
CircuitBreaker - waitDurationInOpenState	30s	Tất cả CB	CB ở trạng thái OPEN trong 30s
CircuitBreaker - slidingWindowSize	10	Tất cả CB	Tính tỷ lệ fail trên 10 request gần nhất
Retry - maxAttempts	3	cart, order	Retry tối đa 3 lần
Retry - waitDuration	500ms	cart, order	Chờ 500ms giữa các lần retry
Retry - retryExceptions	ConnectException, SocketTimeoutException	Tất cả Retry	Chỉ retry network errors, không retry 4xx
TimeLimiter - timeoutDuration	3s	order-service	Timeout khi gọi payment-service

11.2 Error Response chuẩn (từ GlobalExceptionHandler)

Exception	HTTP Status	Error Code	Message
AppException(USER_NOT_FOUND)	404	4001	User not found
AppException(PRODUCT_NOT_FOUND)	404	4002	Product not found
AppException(INVALID_TOKEN)	401	4010	Token is invalid or expired
AppException(UNAUTHORIZED)	403	4030	You do not have permission
AppException(STOCK_INSUFFICIENT)	400	4005	Insufficient stock for this item
MethodArgumentNotValidException	400	4000	Validation failed: {field errors}
ServiceUnavailableException	503	5030	Downstream service is unavailable
CallNotPermittedException (CB open)	503	5031	Service temporarily unavailable
Exception (generic)	500	5000	Internal server error

11.3 Logging Strategy

Level	Dùng khi nào	Ví dụ
DEBUG	Chi tiết flow, SQL queries (chỉ dev)	Entering method createOrder with params...
INFO	Business events quan trọng	Order created: orderId={}, userId={}, total={}
WARN	Unexpected nhưng recoverable	Product service retry attempt 2/3 for variantId={}

Level	Dùng khi nào	Ví dụ
ERROR	Exception, lỗi nghiêm trọng	Payment failed for orderId={}, error={}

12. Lộ Trình Phát Triển (Roadmap)

12.1 Phase 1: Core Foundation (Tuần 1-3)

Tuần	Task	Deliverable	Phụ thuộc
1	Setup parent POM, common-library, Docker infrastructure	Cấu trúc project, common-library compile, PostgreSQL containers chạy	Không
1	identity-service: register, login, JWT RSA	POST /auth/register + /auth/login trả JWT, JWKS endpoint	common-library, postgres-identity
2	api-gateway: setup routes + JWT filter	Gateway route mọi service, reject invalid JWT	identity-service
2	user-service: profile + address CRUD	GET/PUT /users/me, address endpoints với JWT auth	identity-service, api-gateway
3	product-service: catalog full	CRUD product, variant, category + pagination + Swagger UI	common-library
3	file-service: upload ảnh sản phẩm	POST /files trả URL, GET /files/{name} public	Không

12.2 Phase 2: Commerce Core (Tuần 4-6)

Tuần	Task	Deliverable	Phụ thuộc
4	cart-service: add/update/remove items + Resilience4j	Cart API hoàn chỉnh, Circuit Breaker với product-service	product-service
4	Integrate file-service với product-service	Upload ảnh khi tạo/edit product	file-service, product-service
5	order-service: checkout flow	POST /orders từ cart, reserve stock, gọi payment-service	cart-service, product-service, payment-service
5	payment-service: initiate + webhook	Tạo payment session, nhận webhook, update order status	order-service
6	notification-service: email khi order	Gửi email xác nhận đơn hàng	order-service
6	End-to-end test: register → buy → pay	Full happy path qua API Gateway chạy thành công	Tất cả service Phase 1+2

12.3 Phase 3: Production Ready (Tuần 7-9)

Tuần	Task	Deliverable
7	Hoàn thiện Swagger/OpenAPI cho tất cả service	Aggregate Swagger UI tại gateway /swagger-ui.html
7	Flyway migration review + seed data	Migration scripts clean, có dữ liệu test
8	Hardening Resilience4j (timeout, bulkhead)	Cấu hình production-ready cho mọi service-to-service call
8	Security review: RBAC, input validation, SQL injection	Tất cả endpoint validate input, roles check đúng

Tuần	Task	Deliverable
9	Docker Compose production config	Health checks, restart policy, resource limits, volumes cho DB
9	Load test + performance tuning	Identify bottleneck, tune connection pool, JVM flags

12.4 Phase 4: Advanced (Tùy chọn, Tuần 10+)

Feature	Tech	Mô tả
Event-driven	Apache Kafka	Order created → inventory reserved → notification sent (async)
Outbox Pattern	PostgreSQL + polling	Đảm bảo event được publish dù service crash giữa chừng
Saga Pattern	Choreography Saga	Distributed transaction: order + payment + inventory với compensating transactions
Service Discovery	Spring Cloud Eureka	Dynamic service URL thay vì hard-code trong config
Distributed Tracing	Micrometer + Zipkin/Jaeger	Trace request xuyên suốt các service
Centralized Config	Spring Cloud Config Server	Quản lý config tập trung, hot reload
Search	Elasticsearch	Full-text search sản phẩm, filter nâng cao
Cache	Redis	Cache product catalog, cart, giảm DB load
Admin Dashboard	React + Ant Design Pro	Quản lý order, sản phẩm, người dùng

13. Cấu Trúc Thư Mục Chi Tiết

13.1 Mỗi service (ví dụ: product-service)

```

product-service/
├── pom.xml
├── Dockerfile
├── src/main/
│   ├── java/com/fashionstore/product/
│   │   ├── ProductServiceApplication.java
│   │   ├── controller/
│   │   │   ├── ProductController.java
│   │   │   └── CategoryController.java
│   │   ├── service/
│   │   │   ├── ProductService.java
│   │   │   └── CategoryService.java
│   │   ├── repository/
│   │   │   ├── ProductRepository.java
│   │   │   └── CategoryRepository.java
│   │   ├── entity/
│   │   │   ├── Product.java
│   │   │   ├── ProductVariant.java
│   │   │   └── Category.java
│   │   ├── dto/
│   │   │   ├── request/ (CreateProductRequest.java, ...)
│   │   │   └── response/ (ProductDto.java, ...)
│   │   ├── mapper/
│   │   │   └── ProductMapper.java (@Mapper)
│   │   ├── client/
│   │   │   └── FileServiceClient.java
│   │   ├── config/
│   │   │   ├── SecurityConfig.java
│   │   │   └── OpenApiConfig.java
│   │   └── exception/ (nếu có exception riêng)
│   └── resources/
│       ├── application.yml
│       ├── application-docker.yml
│       └── db/migration/
│           ├── V1__init_product.sql
│           └── V2__add_product_images.sql

```

13.2 Application.yml template (Docker profile)

```

# application-docker.yml (product-service example)
spring:
  datasource:
    url: jdbc:postgresql://postgres-product:5434/product_db
    username: ${DB_USERNAME:postgres}
    password: ${DB_PASSWORD:postgres}
  jpa:
    hibernate.ddl-auto: validate
    open-in-view: false
  flyway:
    enabled: true
    locations: classpath:db/migration
  security.oauth2.resourceserver.jwt:
    jwk-set-uri: http://identity-service:8081/.well-known/jwks.json

service:

```



```
file.url: http://file-service:8088

resilience4j.circuitbreaker.instances.fileService:
  registerHealthIndicator: true
  slidingWindowSize: 10
  failureRateThreshold: 50
  waitDurationInOpenState: 30s

management.endpoints.web.exposure.include: health,info,metrics
springdoc.api-docs.path: /v3/api-docs
```

14. Checklist Hoàn Thành

14.1 Infrastructure & Setup

#	Hạng mục	Trạng thái
1	Parent POM với dependency management và plugin config	☐ Todo
2	common-library: ApiResponse, ErrorCode, AppException, GlobalExceptionHandler	☐ Todo
3	common-library: SecurityUtils, DateTimeUtils, AppConstants	☐ Todo
4	Docker Compose: 8 PostgreSQL, 8 service + Gateway	☐ Todo
5	RSA key pair generate: private.pem, public.pem	☐ Todo
6	Flyway migration scripts cho tất cả service	☐ Todo

14.2 Service Completion

#	Service	Endpoints	Tests	Swagger
1	identity-service	☐	☐	☐
2	user-service	☐	☐	☐
3	product-service	☐	☐	☐
4	cart-service	☐	☐	☐
5	order-service	☐	☐	☐
6	payment-service	☐	☐	☐
7	notification-service	☐	☐	☐
8	file-service	☐	☐	☐
9	api-gateway	☐	☐	☐

14.3 Quality Gates

Tiêu chí	Yêu cầu	Trạng thái
Unit Test Coverage	>= 70% per service	☐ Todo
Integration Test	Happy path end-to-end qua Gateway	☐ Todo
API Documentation	Tất cả endpoint có OpenAPI 3 spec đầy đủ	☐ Todo
Security Scan	Không có secret/key trong source code	☐ Todo
Input Validation	Tất cả request DTO dùng Jakarta Validation	☐ Todo
Error Handling	Tất cả exception trả về ApiResponse chuẩn	☐ Todo
Logging	INFO log cho tất cả business event quan trọng	☐ Todo

Tiêu chí	Yêu cầu	Trạng thái
Health Check	/actuator/health UP cho tất cả service	[] Todo
Docker Build	docker compose up --build không lỗi	[] Todo
MapStruct compile	Không có warning trong generate code	[] Todo
Flyway migration	Migration chạy thành công khi startup	[] Todo
Circuit Breaker	CB mở khi downstream down, trả fallback	[] Todo

Fashion Store Microservice - Project Plan v1.0

Được tạo tự động · Tháng 6, 2026 · Java 21 + Spring Boot 3.x + PostgreSQL + Docker